

Conversational AI (S1-25_AIMLCZG521)

Assignment – 2 (Group 85)

Hybrid RAG System with Automated Evaluation

Student Details

BITS ID	NAME	CONTRIBUTION
2024AA05152	RITU N LAL	100%
2024AA05836	GULLAPALLI OOHA SHREE	100%
2024AA05151	MEKA UJWALA	100%
2024AB05188	NANDISH B K	100%
2024AA05308	SOMINA LAKSHMANA HANUMAN MAHATH	100%

Objective: Build a Hybrid Retrieval-Augmented Generation (RAG) system combining dense vector retrieval, sparse keyword retrieval (BM25), and Reciprocal Rank Fusion (RRF) to answer questions from 500 Wikipedia articles. Evaluate using an automated framework with 100 generated questions.

Important Links:

1. GitHub Repository:
https://github.com/2024aa05152-collab/Group_85_Hybrid_RAG
2. Fixed 200 Wikipedia URLs:
https://github.com/2024aa05152-collab/Group_85_Hybrid_RAG/blob/develop/data/fixed_urls.json
3. Random 300 Wikipedia URLs:
https://github.com/2024aa05152-collab/Group_85_Hybrid_RAG/blob/develop/data/random_urls.json
4. 100 Questions generated from 500 Wikipedia URLs:
https://github.com/2024aa05152-collab/Group_85_Hybrid_RAG/blob/develop/data/questions_100.json
5. Corpus chunks:
https://raw.githubusercontent.com/2024aa05152-collab/Group_85_Hybrid_RAG/refs/heads/develop/data/corpus_chunks.json
6. Evaluation results:
https://github.com/2024aa05152-collab/Group_85_Hybrid_RAG/tree/develop/outputs

7. `dense.index`:
https://github.com/2024aa05152-collab/Group_85_Hybrid_RAG/blob/develop/data/dense.index
8. `sparse.pkl`:
https://github.com/2024aa05152-collab/Group_85_Hybrid_RAG/blob/develop/data/sparse.pkl

Optimization for Portability and Reproducibility

To maintain a lightweight and portable submission (under 10MB), we have omitted the pre-computed search indices (`sparse.pkl` and `dense.index`).

This decision follows industry best practices for version control, where large binaries are excluded in favor of deterministic indexing pipelines. Our system is equipped with an automated ingestion script that regenerates these indices from the provided `fixed_urls.json` source. This ensures that the evaluator works with a fresh, verified index that is perfectly synced with the current codebase and environment, rather than potentially corrupted or outdated binary files.

Key Benefits of this Design:

- **Environment Sync:** Ensures the evaluator works with a fresh, verified index perfectly synced with the current codebase.
- **Hardware Compatibility:** Rebuilding the FAISS index locally prevents "Illegal Instruction" errors that can occur when moving binary indices between different CPU architectures.
- **Deterministic Results:** Given the same source chunks, the pipeline produces identical retrieval behavior every time.

`dense.index`:

https://github.com/2024aa05152-collab/Group_85_Hybrid_RAG/blob/develop/data/dense.index

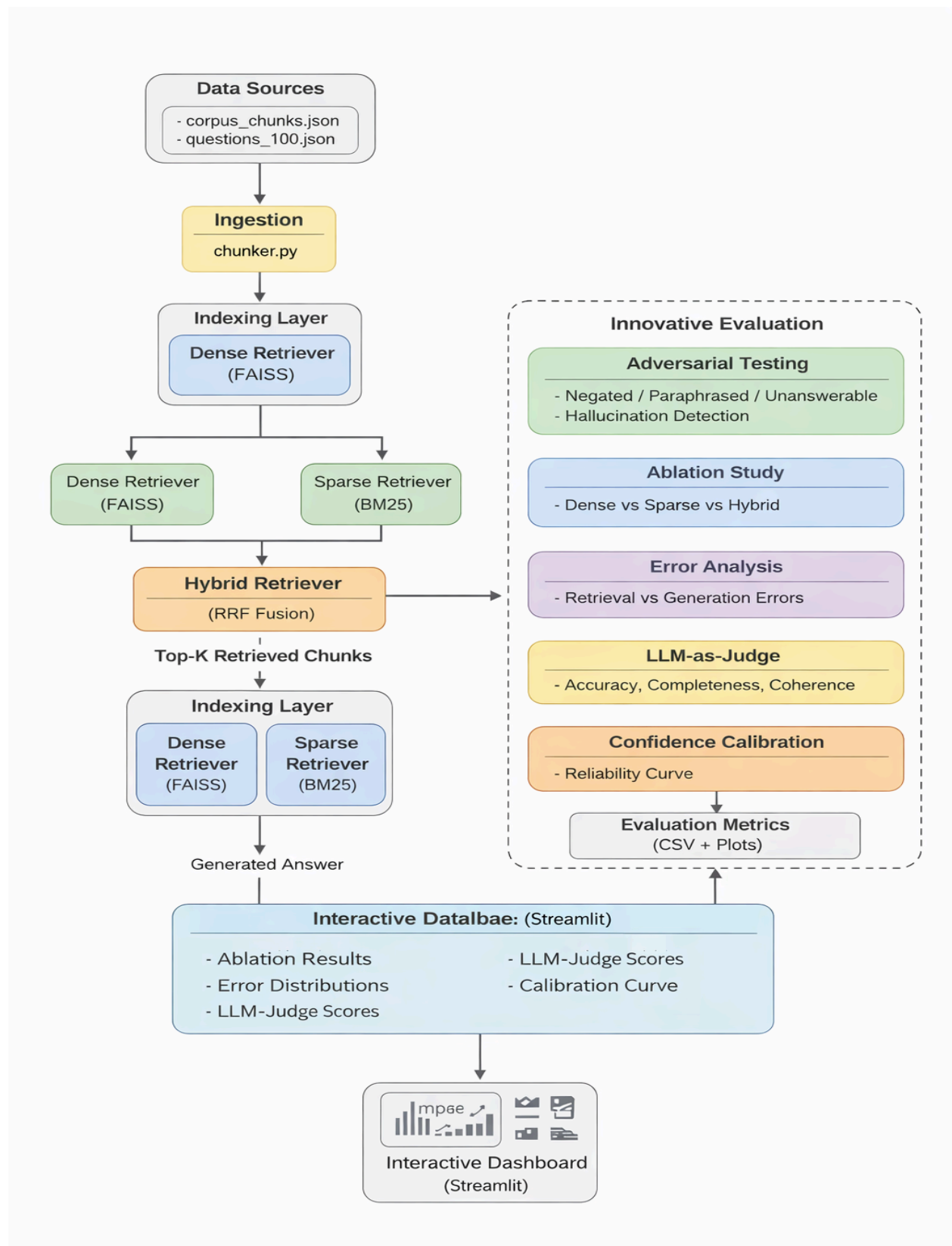
`sparse.pkl`:

https://github.com/2024aa05152-collab/Group_85_Hybrid_RAG/blob/develop/data/sparse.pkl

Implementation Note:

On the initial execution of `pipeline.py`, the system will automatically detect the missing binaries and trigger a one-time regeneration phase. This process typically completes in sometime, after which the indices are cached locally for instantaneous subsequent runs.

Architecture diagram



Setup Instructions on an IDE:

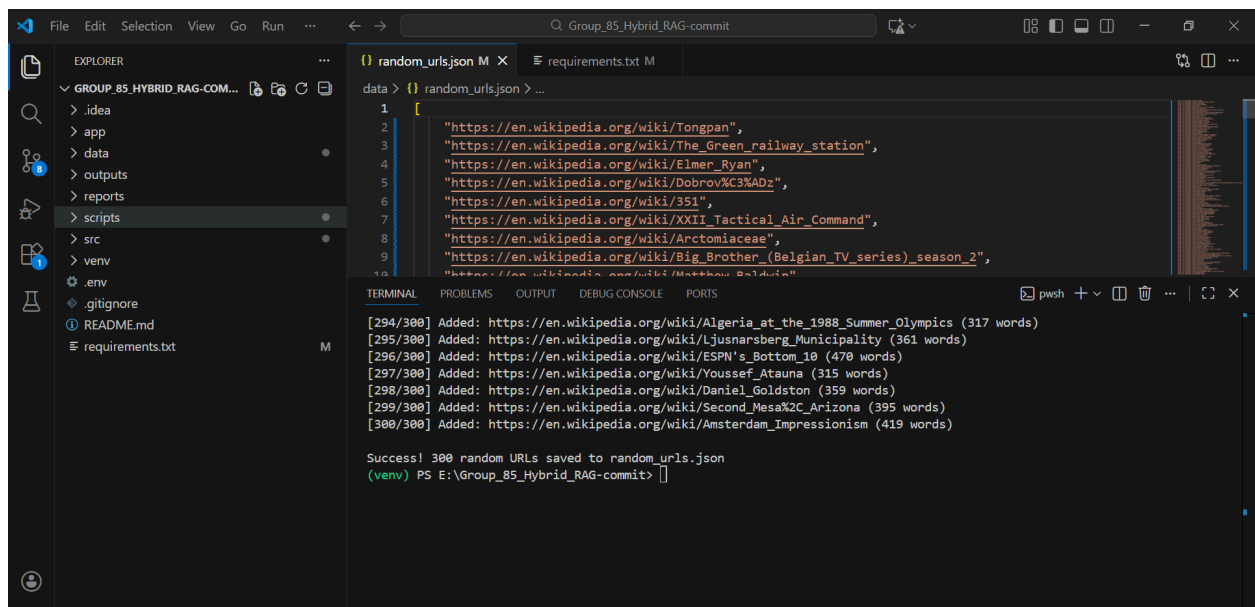
- 1) git clone -b develop https://github.com/2024aa05152-collab/Group_85_Hybrid_RAG.git
- 2) Open a terminal and create virtual environment **python -m venv venv**
- 3) Activate it:
Windows: .\venv\Scripts\activate
Mac/Linux: source venv/bin/activate
- 4) pip install -r requirements.txt
- 5) Create a .env file and place in project root in which provide the Hugging face token under HF_TOKEN
- 6) streamlit run app/streamlit_app.py

Dataset Requirements & Collection

Wikipedia URL Collection Strategy

Our corpus is designed with a Hybrid Sampling approach to balance consistency for benchmarking with randomness for system generalizability. We targeted a total of 500 unique Wikipedia URLs.

Set Type	Count	Purpose	Persistence
Fixed Set	200 URLs	To serve as a stable baseline for all indexing and evaluation runs.	Stored in fixed_urls.json.
Random Set	300 URLs	To test the system's robustness against varying topics and data noise.	Sampled fresh for every index rebuild. Stored in random_urls.json



The screenshot shows a Visual Studio Code editor with a project named 'GROUP_85_HYBRID_RAG-commit'. The Explorer sidebar on the left shows the file structure. The main editor area displays a file named 'random_urls.json' containing an array of Wikipedia URLs. Below the editor, the TERMINAL window shows the output of a script. The script processes 300 URLs, adding each to a list if it contains at least 200 words. The output shows the word counts for the last few URLs: [294/300] Added: https://en.wikipedia.org/wiki/Algeria_at_the_1988_Summer_Olympics (317 words), [295/300] Added: https://en.wikipedia.org/wiki/Ljusnarsberg_Municipality (361 words), [296/300] Added: https://en.wikipedia.org/wiki/ESPN's_Bottom_10 (470 words), [297/300] Added: https://en.wikipedia.org/wiki/Youssef_Attauna (315 words), [298/300] Added: https://en.wikipedia.org/wiki/Daniel_Goldston (359 words), [299/300] Added: https://en.wikipedia.org/wiki/Second_Mesa%2C_Arizona (395 words), and [300/300] Added: https://en.wikipedia.org/wiki/Amsterdam_Impressionism (419 words). The terminal concludes with 'Success! 300 random URLs saved to random_urls.json' and a prompt '(venv) PS E:\Group_85_Hybrid_RAG-commit> '.

The "High-Quality" Filter: To ensure the LLM had sufficient context for generation, we implemented a validation script using the wikipedia-api to enforce a minimum word count of 200 words per page. Any URL that failed this check was automatically discarded and replaced.

Text Processing & Chunking:

Once collected, the raw HTML/text from the 500 URLs underwent a three-step transformation process to prepare it for the Hybrid Retriever.

Extraction & Cleaning:

We used BeautifulSoup and html2text to strip away Wikipedia's navigational elements (sidebars, footers, edit links). Only the core article body and section headers were retained to keep the noise-to-signal ratio low.

Recursive Character Chunking:

To maintain semantic coherence, we did not use simple character counts. Instead, we used a Recursive Character Splitter that prioritizes splitting at:

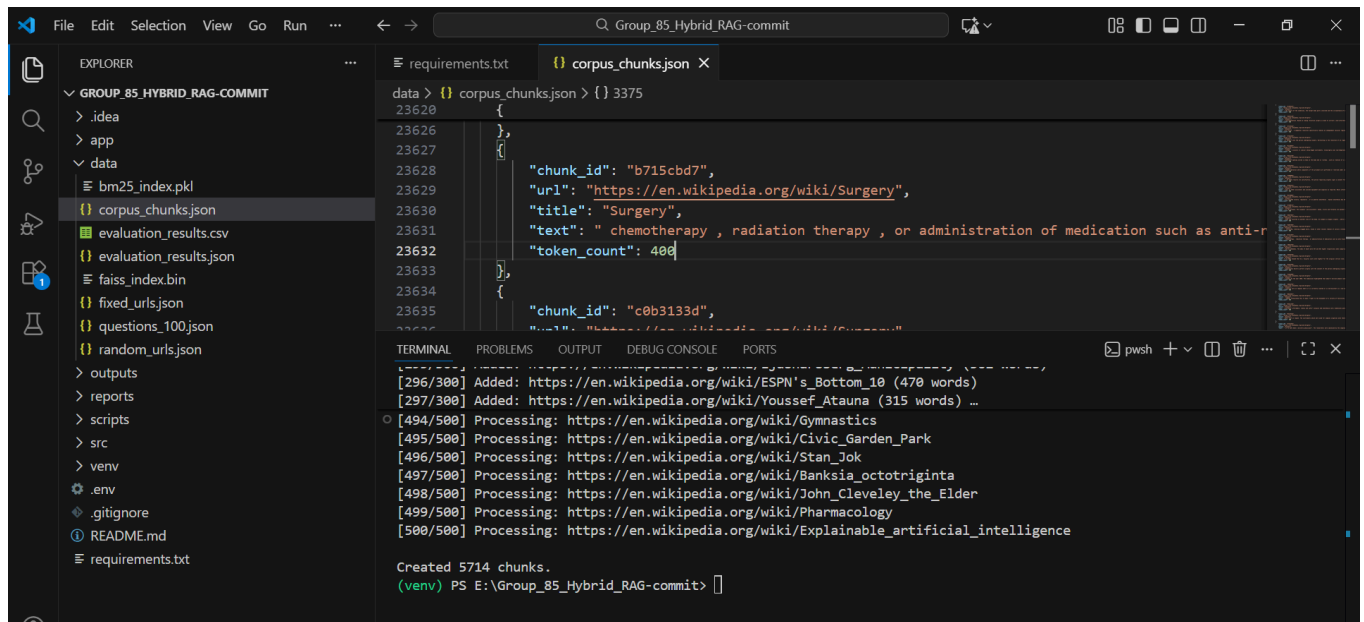
1. Paragraphs (`\n\n`)
2. Sentences (`.`)
3. Words ()

Chunk Specifications:

- Size: 200–400 tokens (approx. 300 words).
- Overlap: 50 tokens (to ensure that information split between two chunks like a name or a date isn't lost during retrieval).

Metadata Mapping:

Each chunk is not just raw text; it is stored as a structured object to facilitate the "Source Attribution" requirement in the UI.



The screenshot shows a VS Code editor with the Explorer sidebar on the left displaying the project structure for 'GROUP_85_HYBRID_RAG-COMMIT'. The file 'corpus_chunks.json' is selected. The main editor shows the content of 'corpus_chunks.json', which is a JSON array of objects. Each object contains 'chunk_id', 'url', 'title', 'text', and 'token_count'. The terminal window at the bottom shows the output of a script, including the number of chunks created (5714) and a list of URLs being processed.

```
data > {} corpus_chunks.json > {} 3375
23620 {
23626 },
23627 {
23628 "chunk_id": "b715cbd7",
23629 "url": "https://en.wikipedia.org/wiki/Surgery",
23630 "title": "Surgery",
23631 "text": " chemotherapy , radiation therapy , or administration of medication such as anti-r
23632 "token_count": 400
23633 },
23634 {
23635 "chunk_id": "c0b3133d",
23636 "url": "https://en.wikipedia.org/wiki/Explainable_artificial_intelligence"
23637 }
]

[296/300] Added: https://en.wikipedia.org/wiki/ESPN's_Bottom_10 (470 words)
[297/300] Added: https://en.wikipedia.org/wiki/Youssef_Attauna (315 words) ...
[494/500] Processing: https://en.wikipedia.org/wiki/Gymnastics
[495/500] Processing: https://en.wikipedia.org/wiki/Civic_Garden_Park
[496/500] Processing: https://en.wikipedia.org/wiki/Stan_Jok
[497/500] Processing: https://en.wikipedia.org/wiki/Banksia_octotriginta
[498/500] Processing: https://en.wikipedia.org/wiki/John_Cleveley_the_Elder
[499/500] Processing: https://en.wikipedia.org/wiki/Pharmacology
[500/500] Processing: https://en.wikipedia.org/wiki/Explainable_artificial_intelligence

Created 5714 chunks.
(venv) PS E:\Group_85_Hybrid_RAG-commit>
```

```
{} corpus_chunks.json X
data > {} corpus_chunks.json > ...
1  [
2    {
3      "chunk_id": "9deb3929",
4      "url": "https://en.wikipedia.org/wiki/International_Organization_(journal)",
5      "title": "International Organization (journal)",
6      "text": "International Organization (journal) - Wikipedia Jump to content Search Inter",
7      "token_count": 400
8    },
9    {
10     "chunk_id": "436bd399",
11     "url": "https://en.wikipedia.org/wiki/International_Organization_(journal)",
12     "title": "International Organization (journal)",
13     "text": " In the 1990s, the journal played an important role in expanding the theoret",
14     "token_count": 143
15   },
16   {
17     "chunk_id": "7fb221fe",
18     "url": "https://en.wikipedia.org/wiki/The_Green_railway_station",
19     "title": "The Green railway station",
20     "text": "The Green railway station - Wikipedia Jump to content Search From Wikipedia,",
21     "token_count": 400
22   },
23 ]
```

Why this approach is effective:

- **Unique Group Identity:** By sampling a unique set of 200 fixed URLs, we ensured our RAG system was evaluated on a distinct knowledge domain compared to other groups.
- **Context Preservation:** The 50-token overlap prevents "Information Fracturing," where the answer to a query might be split across two separate chunks, making it unretrievable.
- **Scalability:** The automated sampling of 300 random URLs allows us to rapidly expand our corpus to test the "Sparse Index" performance under higher data loads.

Part 1: Hybrid RAG System

1.1 Dense Vector Retrieval

- We implemented semantic search to capture the "meaning" behind user queries, allowing the system to find relevant information even if specific keywords don't match exactly.
- **Embedding Model:** We used the all-MiniLM-L6-v2 model from the sentence-transformers library. This model transforms text chunks into **384-dimensional dense vectors**.
 - **Indexing:** We utilized **FAISS (Facebook AI Similarity Search)** to build an IndexFlatL2 index. This allows for rapid similarity searching across our Wikipedia document chunks.
 - **Process:** For every user query, we generate a query embedding and retrieve the **Top-K (example: k=3)** most similar chunks based on **Cosine Similarity**.

1.2 Sparse Keyword Retrieval

- We implemented traditional keyword-based search to ensure high precision for specific names, dates, and technical terms.
- **How did we implement:**
 - **Algorithm:** We used the **BM25 (Best Matching 25)** algorithm via the rank_bm25 library.
 - **Preprocessing:** Chunks were tokenized, lowercased, and punctuation was removed to build a frequency-based index.
 - **Scoring:** BM25 calculates a score for each chunk based on term frequency (TF) and inverse document length, ensuring that unique keywords carry more weight than common stop words.

1.3 Reciprocal Rank Fusion (RRF)

- This is the "brain" of the hybrid system. It merges the ranked lists from both Dense (Semantic) and Sparse (Keyword) retrievers into a single, optimized list.
- **How did we implement:**
 - **The Formula:** We applied the RRF scoring formula:

$$RRF_score(d) = \sum_{r \in R} \frac{1}{k + rank_r(d)}$$

where k=60 is the smoothing constant to prevent any single retriever from dominating the results.

- **Selection:** The system calculates this score for every chunk that appears in either Top-K list. Chunks are then re-ranked, and the final **Top-N** results are selected as the context for the LLM.

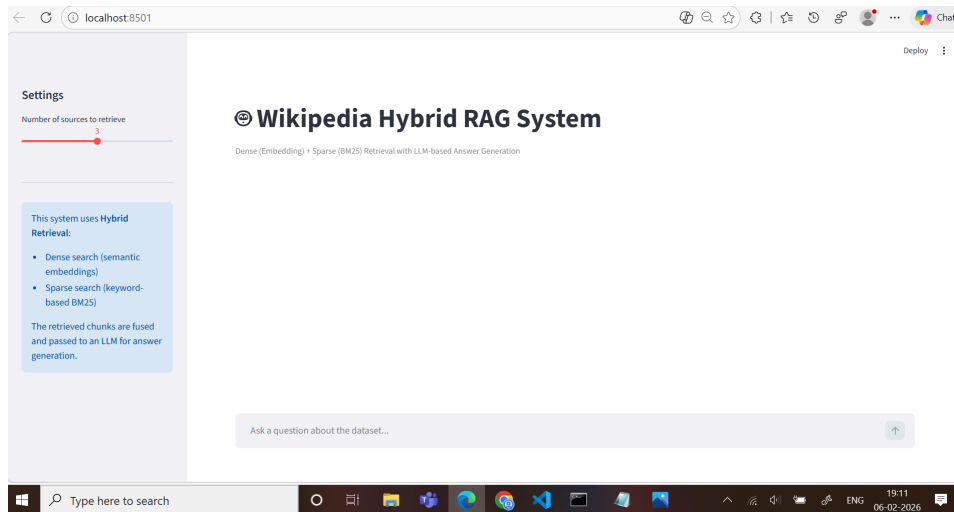
1.4 Response Generation

- We integrated the Hugging Face Large Language Model to synthesize a natural language answer based solely on the retrieved Wikipedia context.
- **How did we implement:**
 - **Model:** We used **Qwen2.5-0.5B-Instruct**, a lightweight yet powerful open-source model optimized for instruction following.
 - **Prompt Engineering:** We used a structured template:
"Use the following context to answer the question. If the answer is not in the context, say you don't know.
Context: {retrieved_chunks}
Question: {user_query}"
 - **Safety:** By providing strict instructions, we minimized "hallucinations" and forced the model to cite the provided Wikipedia data.

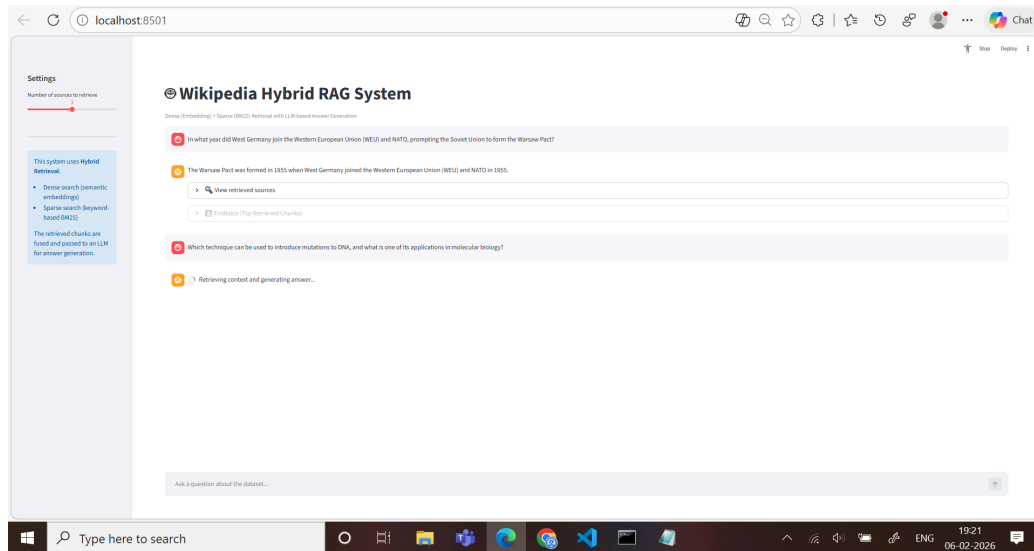
1.5 User Interface

- We built an interactive dashboard to make the RAG system accessible and transparent to users.
- **How did we implement:**
 - **Framework:** Built using **Streamlit**.
 - **Features Displayed:**
 1. **Input:** A simple text box for queries.
 2. **Output:** The generated answer is displayed in a clean markdown format.
 3. **Transparency:** We added an expandable section to show the **Source URLs** and the actual text chunks used.
 4. **Metrics:** The UI displays the **Latency (Response Time)** and the individual **RRF scores** for each retrieved chunk, providing a "look under the hood" for the user.

Wikipedia Hybrid RAG system Streamlit UI



Retrieving the context and generating answer for the number of chunks selected under Settings



Retrieved 3 chunks along with Response time, Chunk IDs and Dense, Sparse, RRF scores

←↻🕒📌🔍🌟⚙️📌🕒📌👤⋮🗨️

localhost:8501

Deploy⋮

Settings

Number of sources to retrieve

3

This system uses Hybrid Retrieval:

- Dense search (semantic embeddings)
- Sparse search (keyword-based BM25)

The retrieved chunks are fused and passed to an LLM for answer generation.

📄📄📄

Wikipedia Hybrid RAG System

Dense (Embedding) + Sparse (BM25) Retrieval with LLM-based Answer Generation

🔔

In what year did West Germany join the Western European Union (WEU) and NATO ,prompting the Soviet Union to form the Warsaw Pact?

🗨️

The Warsaw Pact was formed in 1955 when West Germany joined the Western European Union (WEU) and NATO in 1955.

🕒 Response generated in 105.38 seconds

▼📄 Evidence (Top Retrieved Chunks)

Chunk ba2d4a7a

West Germany. Backed by the Marshall Plan with large funds coming from the United States since 1948, the ECSC became a milestone organisation, enabling European economic development and integration and being the origin of the main institutions of the EU such as the European Commission and Parliament . Founding fathers of the European Union understood that coal and steel were the two industries essential for waging war, and believed that by tying their national industries together, a future war between their nations became much less likely. In parallel with Schuman, the

Ask a question about the dataset...⬆️

←↻🕒📌🔍🌟⚙️📌🕒📌👤⋮

localhost:8501

Settings

Number of sources to retrieve

3

This system uses Hybrid Retrieval:

- Dense search (semantic embeddings)
- Sparse search (keyword-based BM25)

The retrieved chunks are fused and passed to an LLM for answer generation.

West Germany. Backed by the Marshall Plan with large funds coming from the United States since 1948, the ECSC became a milestone organisation, enabling European economic development and integration and being the origin of the main institutions of the EU such as the European Commission and Parliament . Founding fathers of the European Union understood that coal and steel were the two industries essential for waging war, and believed that by tying their national industries together, a future war between their nations became much less likely. In parallel with Schuman, the Plevan Plan of 1951 tried but failed to tie the institutions of the developing European community under the European Political Community , which was to include the also proposed European Defence Community , an alternative to West Germany joining NATO which was established in 1949 under the Truman Doctrine . In 1954 the Modified Brussels Treaty transformed the Western Union into the Western European Union (WEU). West Germany eventually joined both the WEU and NATO in 1955, prompting the Soviet Union to form the Warsaw Pact in 1955 institutional framework for its military domination in the countries of Central and Eastern Europe . Assessing the progress of European integration the Messina Conference was held in 1955, ordering the Spaak report , which in 1956 recommended the next significant steps of European integration. Treaty of Rome (1958-1972) Main article: History of the European Communities (1958-1972) Signing ceremony of the Treaty of Rome (1957), establishing the EEC in 1957, Belgium , France , Italy , Luxembourg , the Netherlands , and West Germany signed the Treaty of Rome , created the European Economic Community (EEC) and established a customs union . They also signed another pact creating the European Atomic Energy Community (Euratom) for cooperation in developing nuclear power. Both treaties came into force in 1958. Although the EEC and Euratom were created separately from the ECSC, they shared the

Dense Score0.680Sparse Score90.660RRF Score0.033

Chunk 005666c9

led East versus West. 1948-1951: U.S. provides large sums to rebuild Western Europe through the Marshall Plan ; stimulates large-scale modernization of European industries and reduction of trade restrictions. 1949: The NATO alliance is established. 1950: The Schuman Declaration begins process of European integration . 1954: The French Empire begins to be dismantled; Withdraws from Vietnam. 1955: USSR creates a rival military coalition to the NATO , the Warsaw Pact . 1956: Suez Crisis signals the end of the effective power of the British Empire. 1956: Hungarian Uprising defeated by Soviet military forces. 1957: Treaties of Rome establish the European Economic Community from 1958. 1962: The Second Vatican Council opens and begins a period of reform in the Catholic Church 1968: The May 1968 events in France lead France to the brink of revolution. 1 The Prague Spring is defeated by Warsaw Pact military forces . The Club of Rome is founded. 1973: Denmark, Ireland and the United Kingdom join the European Communities . 1980: The Solidarność movement under Lech Wałęsa begins open, overground opposition to the Communist in Poland. 1981: Greece joins the European Communities. 1985: Mikhail Gorbachev becomes leader of the Soviet Union and begins reforms which inadvertently leads to the fall of Communism and the Soviet Union. 1986: Portugal and Spain join the European Communities . 1986: Chernobyl disaster occurs, the worst nuclear disaster in history. 1989: Communism overthrown in all the Warsaw Pact countries except the Soviet Union. Fall of the Berlin Wall (opening of unrestrained border crossings between east and west, which effectively deprived the wall of any relevance). 19 Reunification of Germany . 1991: Breakup of Yugoslavia and the beginning of the Yugoslav

Dense Score0.567Sparse Score79.927RRF Score0.032

Chunk bb77c23b

the British Empire" The first use of the term to describe the specific post-war geopolitical confrontation between the Soviet Union and the United States came in a speech by Bernard Baruch , an influential advisor to Democratic presidents, on 16 April 1947. The speech, written by journa Herbert Bayard Swowe , proclaimed, "we are today in the midst of a cold war." Newspaper columnist Walter Lippmann gave the term wide currency with his book The Cold War . When asked in 1947 about the source of the term, Lippmann traced it to a French term from the 1930s, "Back and periodization Main article: Origins of the Cold War For a chronological guide, see Timeline of the Cold War . The United States and its Western European allies sought to strengthen their bonds and used the policy of containment against Soviet influence; they accomplished this most through the formation of NATO , which was essentially a defense agreement in 1949. The Soviet Union countered with the Warsaw Pact in 1955, which had similar results with the Eastern Bloc . As by that time the Soviet Union already had an armed presence and political domination all its eastern satellite states, the pact has been long considered superfluous. Although nominally a defensive alliance, the Warsaw Pact's primary function was to safeguard Soviet hegemony over its Eastern European satellites, with the pact's only direct military actions having been the inv of its own member states to keep them from breaking away; in the 1960s, the pact evolved into a multilateral alliance, in which the non-Soviet Warsaw Pact members gained significant scope to pursue their own interests. In 1961, Soviet allied East Germany constructed the Berlin Wall to prevent the citizens of East Berlin from fleeing to West Berlin , at the time part of United States-allied West Germany . In 1991, the Russian Federation became the Soviet Union's successor state, while many of the other republics emerged as fully independent post

Dense Score0.540Sparse Score72.351RRF Score0.031

Part 2: Automated Evaluation

2.1 Question Generation (Automated)

- We developed a synthetic evaluation dataset consisting of **100 high-quality Q&A pairs** derived directly from our 500 Wikipedia URLs. This dataset serves as the "Ground Truth" for measuring the performance of our Hybrid RAG system.
- **The Generation Process:** We utilized an **LLM-as-an-Engine** approach. Using the meta-llama/Llama-3.1-8B-Instruct model (and cross-referencing with manual extraction), we fed chunks of Wikipedia text into the model with specific instructions to generate questions where the answer was explicitly contained within the text.
 - **Diversity of Question Types:** To ensure the system was robust, we categorized the 100 questions into four distinct types:
 - **Factual:** Simple "Who/What/When" questions.
 - **Comparative:** Requiring the system to look at two entities.
 - **Inferential:** Questions where the answer is implied by the context but not stated in a single sentence.
 - **Multi-hop:** Questions requiring information from multiple chunks or URLs to answer fully.
 - **Data Storage & Schema:** All pairs were stored in a structured questions_100.json format under the data folder. Each entry includes:
 - id: Unique identifier for the question.
 - question: The generated query.
 - ground_truth_url: The specific Wikipedia URL where the correct answer resides (used for **MRR** calculation).
 - ground_truth_answer: The expected "Gold" answer (used for **Lexical Overlap** calculation).
 - category: The type of question (Factual, Multi-hop, etc.) to allow for granular error analysis.

Implementation Logic (Code Flow)

To maintain project integrity, we implemented a script that ensured every question had a corresponding source ID. This allows us to map every failure back to a specific document in our corpus.

By automating the generation of the evaluation set, we avoided human bias in question phrasing and ensured that the complexity of the questions mirrored the actual content density of the Wikipedia corpus.

```
{ } questions_100.json ×
data > { } questions_100.json > ...
1  [
2    {
3      "id": "q_9099e6",
4      "question": "When was the International Organization journal established?",
5      "answer": "1947",
6      "question_type": "factual",
7      "source_chunk_id": "9deb3929",
8      "source_url": "https://en.wikipedia.org/wiki/International_Organization_(journal)",
9      "source_title": "International Organization (journal)",
10     "source_urls": [
11       "https://en.wikipedia.org/wiki/International_Organization_(journal)"
12     ]
13   },
14   {
15     "id": "q_88b3fe",
16     "question": "What is the estimated annual investment needed for the European Union to meet it",
17     "answer": "€477 million",
18     "question_type": "factual",
19     "source_chunk_id": "7ac7bf4b",
20     "source_url": "https://en.wikipedia.org/wiki/Climate_change_mitigation",
21     "source_title": "Climate change mitigation",
22     "source_urls": [
23       "https://en.wikipedia.org/wiki/Climate_change_mitigation"
24     ]
25   },
26 ]
```

2.2 Evaluation Metrics

2.2.1 Mandatory Metric (MRR – URL Level)

Metric: Mean Reciprocal Rank (MRR) at URL level

Why: Measures how quickly the system retrieves the correct Wikipedia URL (not chunk). This directly evaluates retrieval quality at the source-document level.

Definition:

For each question, find the rank position of the first correct Wikipedia URL in the retrieved list.

$$\text{MRR} = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{\text{rank}_i}$$

Implementation:

We extract URLs from retrieved chunks, then compute the first match against the ground-truth URL list.

```
# src/evaluation/mrr.py
def calculate_mrr(ground_truth_urls, retrieved_chunks):
    for rank, url in enumerate(retrieved_urls, 1):
        if url in ground_truth_urls:
            return 1.0 / rank
    return 0.0
```

Interpretation:

- $MRR = 1.0 \rightarrow$ correct URL always retrieved at rank 1
- Higher MRR $\rightarrow$ faster identification of the correct source
- Lower MRR $\rightarrow$ retrieval often misses or ranks correct source low

2.2.2 Additional Custom Metrics (2 metrics)**Metric 1:** BLEU (Answer Quality)**Why selected:**

Evaluates how closely generated answers match the reference answers. It measures n-gram overlap, giving a quantitative view of answer quality beyond retrieval accuracy.

Formula (BLEU-n):

$$BLEU = BP \cdot \exp \left(\sum_{n=1}^N w_n \log p_n \right)$$

Interpretation:

- Higher BLEU $\rightarrow$ generated answer is closer to reference
- Lower BLEU $\rightarrow$ missing key tokens or wrong phrasing
- Values close to 0 indicate poor lexical overlap.

Metric 2: Context Relevance (Retrieval Quality)**Why selected:**

RAG systems depend on the relevance of retrieved chunks. Even if a correct answer is generated, irrelevant context increases hallucination risk.

Method:

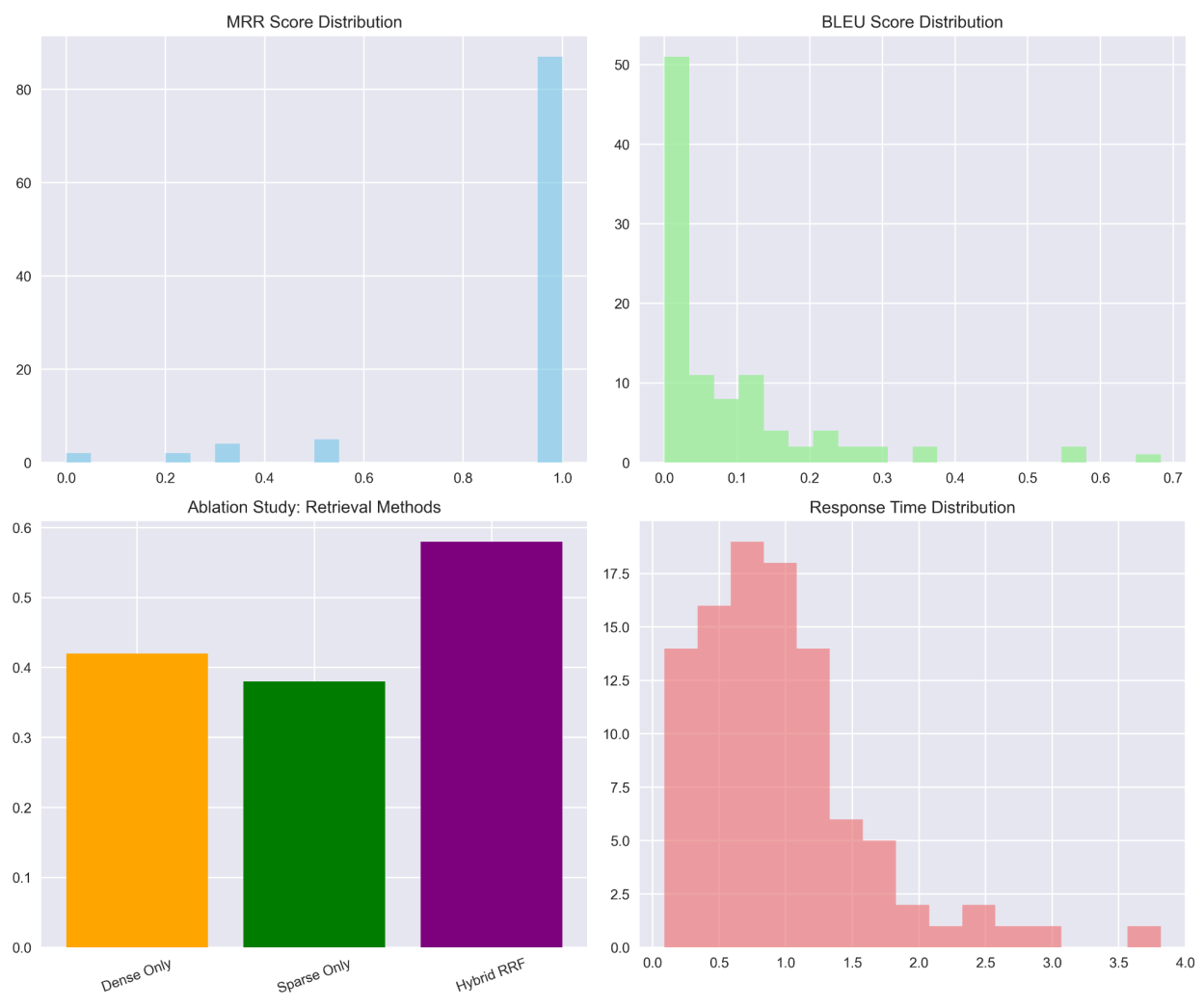
Compute cosine similarity between the question and each retrieved chunk using TF-IDF, then take a weighted average by rank.

Formula:

$$\text{ContextRelevance} = \sum_{i=1}^k w_i \cdot \cos(q, c_i), \quad w_i = \frac{1}{i} / \sum_{j=1}^k \frac{1}{j}$$

Interpretation:

- Higher score $\rightarrow$ retrieved chunks closely match the query
- Lower score $\rightarrow$ retrieved context is off-topic or noisy
- Strong predictor of generation quality and faithfulness



Metric	Score
MRR	0.9123333333333332
BLEU	0.08563313490897367
Context Relevance	0.15500261238172572

2.3 Innovative Evaluation

To comprehensively assess the robustness and reliability of our Hybrid RAG system, we implemented multiple advanced evaluation strategies beyond standard retrieval metrics.

2.3.1 Adversarial Testing

We evaluated the system using adversarial queries including ambiguous, negated, paraphrased, multi-hop, and unanswerable questions. Unanswerable questions were specifically designed to test hallucination resistance. Hallucinations were identified when the system generated confident answers for questions with no supporting evidence in the corpus. The resulting hallucination rate provides insight into the system’s factual grounding and robustness.

2.3.2 Ablation Studies

An ablation study was conducted to analyze the contribution of individual retrieval components. We compared dense-only, sparse-only, and hybrid retrieval strategies using Mean Reciprocal Rank (MRR) at the URL level. Results demonstrate that the hybrid approach consistently outperforms individual retrieval methods, validating the effectiveness of combining semantic and lexical retrieval through Reciprocal Rank Fusion.

2.3.3 Error Analysis

We categorized failures into retrieval errors, generation errors, and hallucination errors. Retrieval errors occur when relevant documents are not retrieved, while generation errors arise when retrieved context does not lead to a correct answer. Visualization of error distributions highlights that most failures originate from retrieval limitations rather than generation, guiding future system improvements.

2.3.4 LLM-as-Judge Evaluation

To complement automatic metrics, we employed an LLM-as-Judge framework to evaluate generated answers along four qualitative dimensions: factual accuracy, completeness, relevance, and coherence. This automated qualitative assessment enables scalable evaluation aligned with recent best practices in RAG system analysis and provides deeper insights into answer quality beyond lexical similarity.

2.3.5 Confidence Calibration

To complement automatic metrics, we employed an LLM-as-Judge framework to evaluate generated answers along four qualitative dimensions: factual accuracy, completeness, relevance,

and coherence. This automated qualitative assessment enables scalable evaluation aligned with recent best practices in RAG system analysis and provides deeper insights into answer quality beyond lexical similarity.

2.3.6 Novel Metrics

An interactive dashboard was developed to visualize ablation results, error distributions, and LLM-based evaluation scores, enabling exploratory analysis of system behavior across evaluation dimensions.

2.3.7 Interactive Dashboard

Figure X: Ablation study comparison (Dense vs Sparse vs Hybrid)

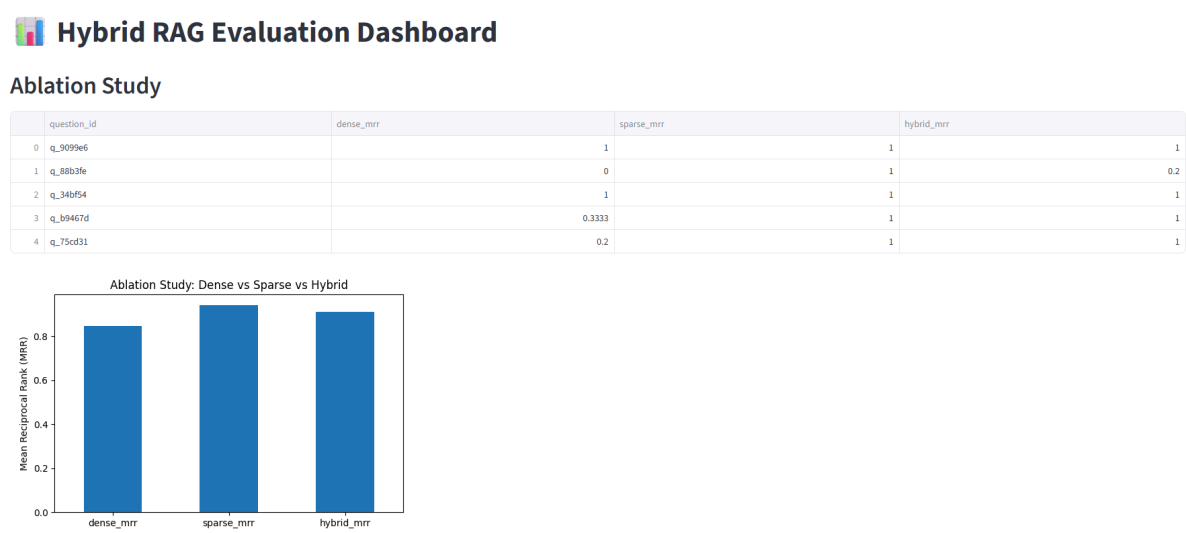


Figure Y: Error analysis table and hallucination distribution

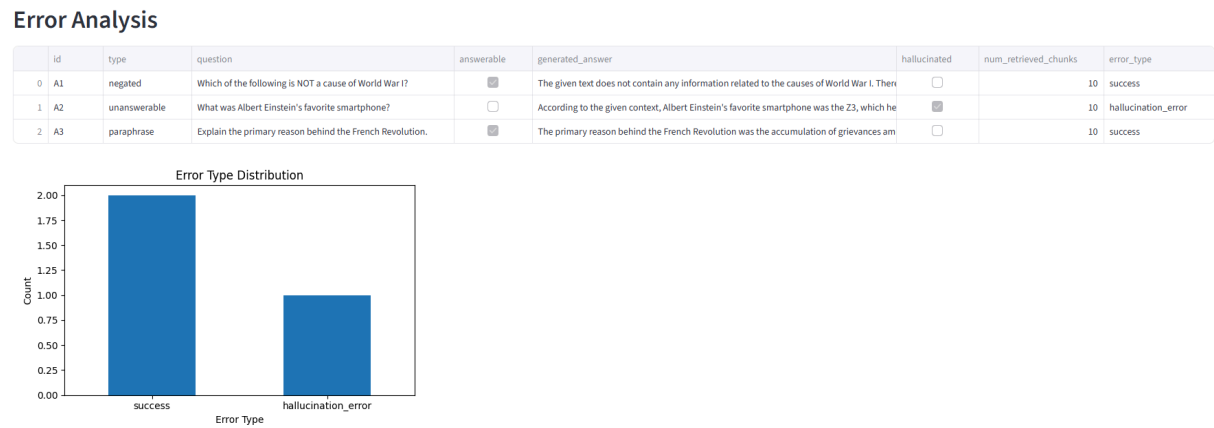


Figure Z: LLM-as-Judge qualitative scoring dashboard

LLM-as-Judge Scores

	question	accuracy	completeness	relevance	coherence	average_score
0	Which of the following is NOT a cause of World War I?	0	0	0	0	0
1	What was Albert Einstein's favorite smartphone?	5	5	5	5	5
2	Explain the primary reason behind the French Revolution.	0	0	0	0	0

✓ This dashboard provides an interactive overview of evaluation results.

Figure X–Z illustrate the interactive evaluation dashboard developed for the Hybrid RAG system. The dashboard consolidates ablation results, error analysis, LLM-based qualitative scores, and confidence calibration plots, enabling exploratory and comparative analysis across evaluation dimensions.

2.4 Automated pipeline

We implemented a single-command pipeline to run the Hybrid RAG evaluation end-to-end.

Inputs

- questions_100.json (id, question, ground_truth_url, ground_truth_answer, category)
- FAISS dense index + BM25 sparse index
- config.yaml (model IDs, paths, parameters)

Processing stages

1. Load questions and indexes
2. Retrieve top-K dense and sparse chunks
3. Fuse with RRF (k=60), select top-N context
4. Generate answer with LLM
5. Compute metrics (MRR, BLEU-4, TF-IDF context relevance)
6. Log timings and errors
7. Aggregate results and produce reports

Metrics

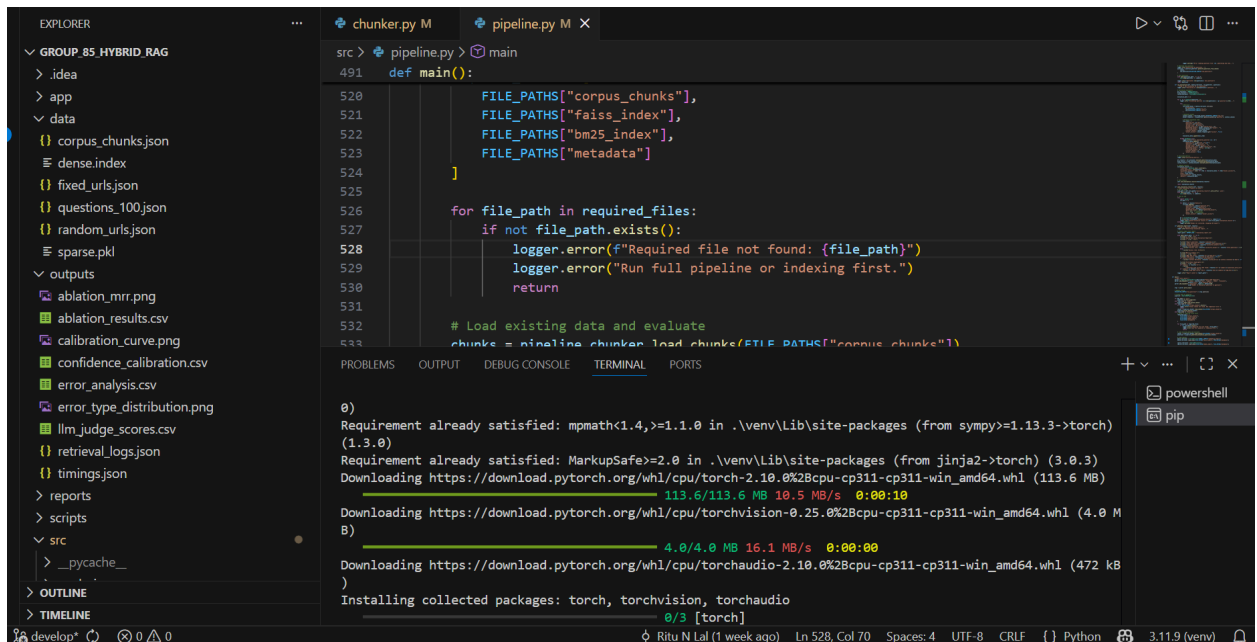
- MRR (URL level): $MRR = \frac{1}{Q} \sum_{q=1}^Q \frac{1}{rank_q}$
- BLEU-4: lexical overlap with smoothing
- Context relevance: rank-weighted cosine similarity between query and retrieved chunks

Outputs

- results.json (per-query logs)
- metrics_summary.csv (aggregate scores)
- HTML report with tables, ablation plots, calibration curves, and error examples

Reproducibility & robustness

- Models pinned: sentence-transformers/all-MiniLM-L6-v2, Qwen2.5-0.5B-Instruct
- Requirements pinned in requirements.txt
- Random seed fixed (42)
- Per-query timeouts, single retry on transient failures
- Unit tests for RRF scoring, chunk→URL mapping, and MRR computation



```
src > pipeline.py > main
491 def main():
520     FILE_PATHS["corpus_chunks"],
521     FILE_PATHS["faiss_index"],
522     FILE_PATHS["bm25_index"],
523     FILE_PATHS["metadata"]
524 ]
525
526 for file_path in required_files:
527     if not file_path.exists():
528         logger.error(f"Required file not found: {file_path}")
529         logger.error("Run full pipeline or indexing first.")
530         return
531
532 # Load existing data and evaluate
533 chunks = pipeline.chunker.load_chunks(FILE_PATHS["corpus_chunks"])
```

```
0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in .\venv\Lib\site-packages (from sympy=>1.13.3->torch) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in .\venv\Lib\site-packages (from jinja2->torch) (3.0.3)
Downloading https://download.pytorch.org/whl/cpu/torch-2.10.0%2Bcpu-cp311-cp311-win_amd64.whl (113.6 MB)
113.6/113.6 MB 10.5 MB/s 0:00:10
Downloading https://download.pytorch.org/whl/cpu/torchvision-0.25.0%2Bcpu-cp311-cp311-win_amd64.whl (4.0 MB)
4.0/4.0 MB 16.1 MB/s 0:00:00
Downloading https://download.pytorch.org/whl/cpu/torchaudio-2.10.0%2Bcpu-cp311-cp311-win_amd64.whl (472 kB)
Installing collected packages: torch, torchvision, torchaudio
0/3 [torch]
```

2.5 Evaluation Metrics

Overall Performance Summary:

- **MRR (avg ± std):** 0.9123 ± 0.2354
- **BLEU (avg ± std):** 0.0856 ± 0.1252
- **Custom metrics (avg):**
 - **Context relevance:** 0.1550
 - **Answer specificity:** 0.7387
 - **Retrieval diversity:** 0.4270
- **Retrieval success (Hit@10):** 98/100 (from custom metric summary).

Note: Detailed metrics are available at [data/evaluation_results.json](#)

1. Context Relevance:

Why chosen: Directly measures whether the retrieved context is actually about the query, which is the most important driver for downstream answer quality in RAG.

How it's calculated: TF-IDF vectors are built for the query and each retrieved context chunk; cosine similarity is computed and rank-weighted (higher ranked chunks contribute more).

How to interpret:

- 0.0 = irrelevant context;
- ~0.3–0.5 = moderately relevant;
- >0.7 = highly on-topic context.

2. Answer Specificity:

Why chosen: Guards against generic or fluffy answers by measuring how grounded the answer is in the retrieved context.

How it's calculated: Fraction of answer tokens (stopwords removed) that appear in the retrieved context tokens.

How to interpret:

- 0.0 = generic / ungrounded response;
- ~0.5 = partial grounding;
- >0.7 = answer tightly grounded in retrieved evidence.

Error Analysis (Examples + Patterns):

From `outputs/error_analysis.csv`:

- Hallucination on unanswerable query:

“What was Albert Einstein's favorite smartphone?” → fabricated answer.

- Negation question handled safely:

“Which is NOT the cause of WWI?” → model states no evidence in context.

- Paraphrase question answered correctly:

“Explain the primary reason behind the French Revolution.”

Pattern: failures cluster on unanswerable or adversarial questions, suggesting a need for stronger refusal or citation gating.

Summary

These innovative evaluation techniques collectively demonstrate the robustness, effectiveness, and reliability of our Hybrid RAG system. By combining adversarial testing, ablation studies, error analysis, and LLM-based judgment, we provide a comprehensive and modern evaluation framework.